

IBM Bob takes an app global

"Bob built this app English-only. Bob made it world-ready."

English + Français (Québec) + العربية (right-to-left)

Live: chinmoypaul8897.github.io/worldready

IBM Bob 2.0 Hackathon · lablab.ai · September 2026

Going global breaks the code, not just the copy

Workflow improved: application maintenance and release — the localization retrofit, plus keeping new code world-ready on every PR.

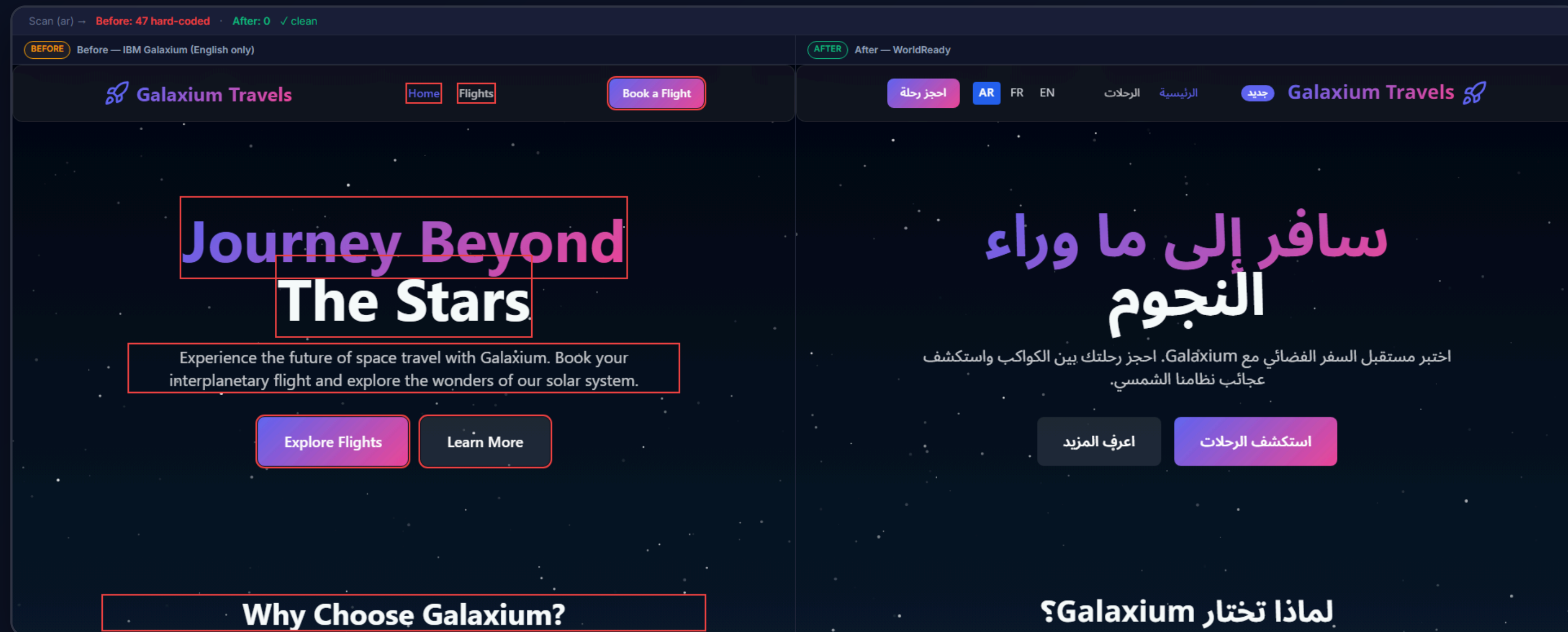
- Entering a new market is a **code problem**, not a translation problem.
- Quebec's **Bill 96** (French) and Gulf/MENA growth (Arabic) demand real localization — Arabic means **right-to-left** layout and Arabic-locale dates, currency and plural forms (**6** of them).
- Machine translation can't fix hard-coded strings, glued plurals ("1 seats left"), physical left/right layout, or English baked into data and toast messages.
- One hard-coded string shipped later **silently breaks** the new market.
- Baseline (measured): **305** hard-coded user-visible strings · **1** language · `<html lang="en">` with no dir · **12** physical left/right classes.

A Bob-driven i18n retrofit — with a gate

- Turns an English-only React app into **English + French (Québec) + Arabic (RTL)**.
- Extracts every user-visible string into **i18next keys** — parallel subagents, one per app area.
- Real i18n, not just words: Intl number/date/relative-time · CLDR plurals (Arabic's **6** forms) · logical RTL layout · mirrored icons · Noto Sans Arabic.
- A **translator round-trip spreadsheet**: a human reviews it, Bob reads it back and applies the edits.
- A **commit hook** and a **CI gate** that block any new hard-coded string — on every PR.

See it flip

- The live viewer: switch to العربية and the whole app mirrors to RTL.
- Scan outlines hard-coded English — Before: 47-48 hard-coded on the home view · After: 0.



chinmoypaul8897.github.io/worldready · live viewer, Arabic selected, Scan: Before 47 · After 0

Built with IBM Bob 2.0

- The product i18n work was done by IBM Bob 2.0 across **19** tasks (**19.41** of 40 Bobcoins); independent verifiers, the demo mock and deploy plumbing were written separately so **Bob never graded itself**.
- Plan mode · custom "**i18n-extractor**" mode (edit scope limited by fileRegex) · custom **i18n-extract** skill · **PreToolUse** commit-gate hook · **GitHub Actions** i18n gate.
- **5** parallel subagents (one per app area) · document understanding (glossary.xlsx + style-guide.pdf) · Office-file write (translator sheet).
- **Translator read-back**: a human reviewed **5** flagged items (**7** cells); Bob read the .xlsx back with its Office tool and applied **7/7**.
- An honest **plain-Bob baseline** (no kit) is committed as a fairness control · prompts, histories and costs in `bob_sessions/`.

Measured, not claimed

- **305** → **0** hard-coded user-visible English strings (independent verifier in `scripts/`).
- **24 / 30** engineered i18n traps fixed — scored against a HELD-OUT key whose SHA-256 was committed before any Bob task ran; no Bob prompt ever named a trap.
- **0** characters of existing English content changed (the language switcher is +45 chars of new UI).
- **key-parity PASS** across en/fr/ar/pseudo · 1 → 3 languages + a pseudo-locale · physical left/right classes 12 → 0 · Arabic plural forms 6/6.
- **Effort: 348** strings extracted into keys in about **12 minutes** of Bob time (five parallel subagents); the whole retrofit took about **40 minutes** of Bob time (translation not timed: an API outage broke its clock).
- The gate is live: a public PR goes **RED** on a hard-coded string, **GREEN** after the i18next fix.

Who pays, and why us

- **Buyers:** product teams entering regulated/RTL markets (Quebec Bill 96, Gulf/MENA); agencies retrofitting client apps.
- **Alternatives** — Lingo.dev, i18next-parser, Crowdin — help extract or translate; none retrofit the code traps (plurals, formatting, RTL, data & toasts) AND install a regression gate in one pass.
- **Differentiation:** TransCreate localizes developer videos; WorldReady localizes app code — keys, plurals, formatting, RTL, and a gate. Accessibility tools such as Usher and CurbCut ask whether everyone can use the app; WorldReady asks whether every market can.

Honest limits

- An early commit (918222d) held the trap list for **~3 minutes, before any Bob task ran**; no Bob prompt referenced it; the score was frozen before the key was published; the plain-Bob baseline is the fairness control.
- **24/30** is honest: the **6** remaining traps (error/format paths off the happy path) were **never answer-fed**.
- Translations are **machine drafts**; a human reviewed **5** flagged items (7 cells) and Bob applied them via the read-back.
- Claude Code made **2** small bug-fixes to Bob's viewer and **2** trivial TypeScript annotations so Bob's code compiled; it pre-diagnosed some small Bob fix tasks (stated in the prompts).
- The demo PR #1 commits were **made by hand** to demonstrate the gate, not by Bob.
- The video narration is an **AI-generated voice** (Microsoft neural TTS).

Next: more locales · RTL visual-regression in CI · a hosted "paste your repo" scan.

Live: chinmoypaul8897.github.io/worldready
Repo: github.com/chinmoypaul8897/worldready

Built with IBM Bob 2.0.